

TECHMASTER · MASTERS OF RECYCLING

e-Pod

Zero-Waste Wireless Audio Player — Build & Launch Plan

28 days

to event day

Solo build

ReSpeaker Lite + SSD1306 OLED

2026-08-02 → 2026-08-29

plan window

Finish by Day 24

(2026-08-25), 4-day buffer

Contents

1. Project Overview & Winning Narrative
2. Bill of Materials — Buy vs. Harvest
3. System Architecture & Wiring
4. Firmware Feature Roadmap
5. Mobile App Spec (Android Studio / Kotlin / BLE)
6. 28-Day Schedule
7. Documentation & YouTube Video Plan
8. Winning Strategy — Metrics, Display & Judge Pitch
9. Event-Day Checklist

Assumptions baked into this plan (edit if wrong): event is exactly 28 days out from 2026-08-01 → event day = **2026-08-29**. You're building **solo**, the ReSpeaker Lite / harvested speaker & battery / MicroSD module are already **in hand**, and the mobile app will be a **native Android app in Android Studio (Kotlin)** since you're a hardware engineer using AI pair-programming rather than a mobile dev. Display is a standard **0.96" SSD1306 I2C OLED**.

1. Project Overview & Winning Narrative

Tagline: Resurrecting e-waste into a next-gen wireless audio player.

e-Pod extracts working components from dead electronics — a laptop speaker, a modem's lithium battery, a remote control's buttons — wires them into a ReSpeaker Lite (ESP32-S3) core, and houses everything in a 3D-printed shell made from recycled plastic. It plays music from a salvaged MicroSD reader, syncs wirelessly over Bluetooth Low Energy with a companion Android app (no charging cable required for song transfer), and layers in judge-pleasing extras: an OLED status screen, offline voice control ("Hey e-Pod, next track"), a beat-synced RGB visualizer, and a voice-memo recorder.

Why this design wins

Judging criterion	How e-Pod delivers
Recycled material compliance	Speaker, battery, buttons, wiring/switch, and heatsink accents are salvaged; the case is printed in recycled PETG/PLA.
Precision of execution	Clean BLE control + file-transfer protocol, working OLED UI, stable audio pipeline — a finished product, not a breadboard demo.
Innovation	Offline voice control and BLE-only file sync (no cable, no cloud) are uncommon in a DIY audio player.
Storytelling	Donor-device cards, before/after photos, an exposed "guts window," and a measured waste-diverted metric turn scrap into a narrative.

Locked feature set

MVP

SD playback · BLE remote control · BLE file transfer · OLED now-playing UI · tactile button navigation

Stretch

RGB beat visualizer · "Hey e-Pod" wake-word voice control · voice-memo recorder

2. Bill of Materials — Buy vs. Harvest

2.1 Already in hand confirmed

Item	Role
ReSpeaker Lite (XIAO ESP32-S3)	Core MCU, BLE/Wi-Fi, 5W amp, mic array, RGB LED — see §3
Salvaged laptop speaker (8Ω, 1–2W)	Primary audio output
Salvaged modem Li-ion battery (3.7V)	Rechargeable power cell
MicroSD SPI breakout (WEMOS 1604-style)	Song storage interface
Remote-control tactile buttons	Play/Pause/Next/Prev

2.2 Still to buy / confirm you have

Component	Search query	Why you need it	Est. cost
TP4056 Type-C charger board (<i>with protection</i>)	TP4056 Type-C battery charger protection board	Safely charges the salvaged cell, prevents over-discharge	\$0.50–\$1
0.96" OLED display	SSD1306 0.96 inch I2C OLED 128x64	Now-playing / battery / BLE-status UI	\$2–\$4
Recycled 3D filament	rPETG or rPLA recycled bottle filament	Prints the enclosure from post-consumer plastic	~\$20/kg
JST 2-pin connector (if speaker isn't already JST)	JST PH 2.0mm 2-pin connector	Clean plug into ReSpeaker Lite's speaker header	<\$1
Small toggle/slide switch (<i>harvest first — see 2.3</i>)	—	Main power on/off between battery and TP4056/board	harvest

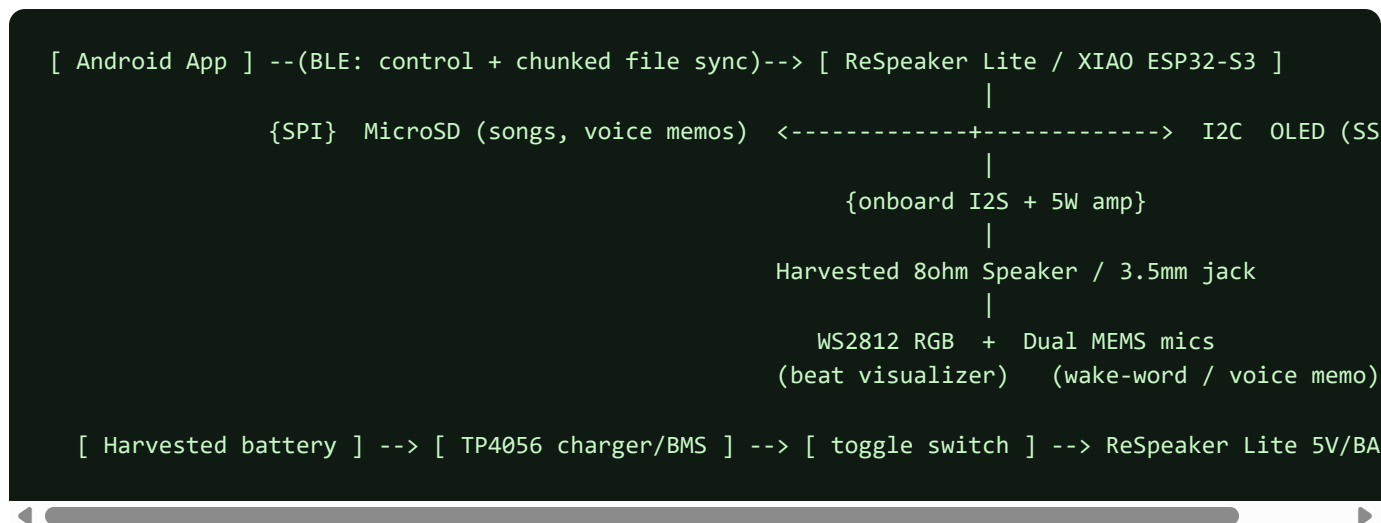
M2 screws / brass inserts (optional)	M2 heat-set inserts 3D printing	Reusable, screw-together enclosure instead of glue	\$3-\$5
--------------------------------------	---------------------------------	--	---------

Order on Day 1. These are the only items on the critical path with shipping lead time — everything else is either in hand or harvested locally. Ordering late is the single biggest risk to the whole 28-day plan.

2.3 Harvest checklist (document with before/after photos)

Part	Donor source	Extraction note
8Ω speaker have	Dead laptop	Already salvaged — verify continuity/impedance before final mount
3.7V Li-ion cell have	Dead modem	Already salvaged — bench-test voltage & a full charge/discharge cycle before trusting it in the final build
3–4 tactile buttons	Broken remote / router / DVD faceplate	Desolder 2-pin micro switches
Power toggle switch + hookup wire	Old PC PSU or printer wiring loom	Reuse for battery→TP4056→board path
Copper heatsink / RF shield can (<i>optional aesthetic</i>)	Laptop cooling assembly, router shield	Mount behind the enclosure's clear window as a visible "salvaged metal" accent

3. System Architecture & Wiring



3.1 ReSpeaker Lite — capability recap

Subsystem	Capability
Compute	ESP32-S3R8 dual-core @ 240MHz, 8MB PSRAM, 8MB flash, plus a dedicated XMOS XU316 audio DSP
Wireless	BLE 5.0 (+ Mesh) for app control/sync; 2.4GHz Wi-Fi available as a fallback fast-transfer path
Audio out	Onboard 5W Class-D amp, 2-pin JST speaker header, 3.5mm headphone jack — no separate DAC board needed
Audio in	Dual PDM MEMS mic array with hardware AEC / noise suppression / AGC — enables wake-word + voice memo features
Power	USB-C or 5V/BAT solder pads; deep-sleep ~14µA for standby battery saving
UI/IO	WS2812 RGB LED, Mute/USR/Boot/Reset buttons, breakout GPIOs for SPI/I2C/UART/PWM/ADC

3.2 MicroSD module (SPI)

SD module pin	XIAO ESP32-S3 pin
---------------	-------------------

VCC	3.3V or 5V
GND	GND
CS	D1
MOSI	D10
MISO	D9
SCK	D8

3.3 SSD1306 OLED (I2C)

Wire to the board's labeled **SDA/SCL** broken-out pins (typically 3.3V, GND, SDA, SCL — 4 wires total). Default I2C address is usually 0x3C.

Verify before soldering: confirm the exact SDA/SCL GPIO numbers against the current Seeed ReSpeaker Lite wiki pinout diagram for your board revision, and make sure they don't collide with the SD-card SPI pins above. Pick two free GPIOs if the silkscreen doesn't label dedicated I2C pins.

3.4 Buttons & power

- **3–4 harvested tactile buttons** → free GPIOs with internal pull-ups enabled in firmware (Play/Pause, Next, Prev, Mode). The onboard **USR** button can double as a 5th function (e.g. toggle Player/Recorder mode); avoid repurposing **BOOT** since it's needed for flashing.
- **Power path:** harvested battery → TP4056 BAT pads → toggle switch → ReSpeaker Lite 5V/BAT pads. TP4056 OUT feeds the board; the board's own regulator handles 3.3V logic.

Battery safety: never puncture or deform the salvaged cell. Confirm it has an intact protection tab/board, bench-test a full charge/discharge cycle on the TP4056 before final assembly, and keep it away from the speaker magnet and any metal accents.

4. Firmware Feature Roadmap

Build MVP first, top to bottom — each item should produce a working demo before moving to the next. Stretch features are ordered by payoff-per-effort; cut from the *bottom* of the stretch list if Day 11 (the built-in catch-up slot) runs out.

MVP must work for the demo

1. List & play MP3/WAV files from the MicroSD card through the onboard amp
2. Play/Pause/Next/Prev via harvested tactile buttons
3. OLED now-playing screen: track name, battery %, BLE connection icon
4. BLE GATT control service: play/pause/volume/next/prev from phone
5. BLE chunked file-transfer service: write incoming song data straight to the SD card

Stretch differentiators

6. WS2812 RGB visualizer — pulse/color-shift driven by audio amplitude
7. Offline "Hey e-Pod" wake-word + voice commands via ESP-SR (next track, play, pause)
8. Voice-memo recorder — mic input to WAV file on SD, played back like a track

5. Mobile App Spec — Android Studio / Kotlin / BLE

Native Android in Kotlin is the right call here: you already have Android Studio set up and are pair-programming with AI, so there's no reason to take on Flutter/React Native's extra learning curve under time pressure. Recommend **Nordic's Android-BLE-Library** (wraps Android's raw BluetoothGatt callback API, which is notoriously fiddly) over hand-rolling BLE from scratch.

Screens

Screen	Function
Scan & Connect	Scans for the e-Pod's advertised BLE service UUID, shows signal strength, connects/pairs
Now Playing / Remote	Play/Pause/Next/Prev/Volume, mirrors the OLED's track info via GATT notifications
Library / File Transfer	Pick an MP3 from phone storage (Storage Access Framework), chunk & stream to the device with a progress bar
Device Status	Battery %, SD storage used/free, BLE connection state

Implementation notes

- **Permissions:** on Android 12+, request `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` at runtime (replaces the old location-permission requirement for BLE scanning).
- **Transfer protocol:** call `requestMtu(517)` after connecting to negotiate a larger packet size, then stream the file in sequenced chunks with a simple ACK so the firmware can detect drops and request a resend.
- **Resilience:** handle reconnect-after-drop gracefully — BLE connections on Android can silently die; add a "reconnecting..." state rather than letting the UI hang.
- **AI-assisted workflow tip:** since this is your first mobile app, build one screen at a time and get each one actually talking to the firmware (test with the free **nRF Connect** app first to validate your GATT service before writing any app code) before layering on UI polish.

6. 28-Day Schedule

Plan window: **2026-08-02** → **2026-08-29** (event day). Build/create work is scheduled to finish by **Day 24 (2026-08-25)**, leaving a 4-day buffer for rehearsal and contingency. **FILM** tags mark days worth shooting B-roll for the YouTube video — do this as you go, not all on Day 24.

Day	Date	Phase	Focus	Exit criteria
1	Aug 2 (Sun)	Harvest & Bring-up	Order TP4056, OLED, filament, JST connector <i>today</i> . Inventory + harvest buttons & switch from a dead remote/PSU. FILM	Order placed; donor devices opened & photographed
2	Aug 3 (Mon)		Bench-test salvaged speaker (continuity/impedance) and battery (voltage, charge cycle on TP4056 once it arrives — or a spare charger meanwhile). FILM	Speaker verified good; battery holds charge
3	Aug 4 (Tue)		Wire ReSpeaker Lite + MicroSD SPI module; flash a basic sketch; format card FAT32 with test MP3s.	Board boots, SD card readable over serial log
4	Aug 5 (Wed)		Get baseline SD→onboard-amp→speaker playback working; test 3.5mm jack.	A test track plays audibly through the harvested speaker
5	Aug 6 (Thu)		Wire & bring up SSD1306 OLED over I2C; render a splash screen alongside playback.	OLED shows text with no SPI/I2C pin conflicts

6	Aug 7 (Fri)	Firmware Core	Wire harvested buttons; implement track list + Next/Prev/Play/Pause.	Buttons control playback
7	Aug 8 (Sat)		OLED now-playing UI v1: track name, battery %, BLE status icon.	Screen updates live during playback
8	Aug 9 (Sun)		BLE GATT control service (play/pause/vol/next/prev); test with nRF Connect app.	Phone (via nRF Connect) can control playback
9	Aug 10 (Mon)		BLE chunked file-transfer service, writes straight to SD; test from a PC/script first. FILM	A file sent over BLE plays back correctly
10	Aug 11 (Tue)		WS2812 RGB beat-sync visualizer.	LED reacts visibly to audio amplitude
11	Aug 12 (Wed)		Catch-up slot. Finish any slipped item above, or start voice-memo recorder if on schedule.	Days 6–10 fully stable

12	Aug 13 (Thu)	Mobile App	New Android Studio project; permissions; BLE scanner screen filtering for e-Pod's UUID.	App lists the e-Pod when nearby
13	Aug 14 (Fri)		Connect/disconnect + wire Play/Pause/Next/Prev buttons to the control characteristic.	App buttons control the device
14	Aug 15 (Sat)		Subscribe to notifications; show track name/battery/BLE status in-app.	App mirrors the OLED screen
15	Aug 16 (Sun)		File picker + chunked BLE transfer with progress bar; negotiate MTU 517.	A song picked on the phone lands on the SD card
16	Aug 17 (Mon)		Polish app UI: device library list, upload queue, storage indicator.	App feels like a finished companion, not a test harness
17	Aug 18 (Tue)		End-to-end integration & bug-fix day: full scan→connect→control→transfer→reconnect loop.	No crashes across 10+ round trips
18	Aug 19 (Wed)	Enclosure	CAD the enclosure (board + battery + speaker + OLED window + button holes + speaker grille + "guts window").	Printable model ready
19	Aug 20 (Thu)		3D print in rPETG/rPLA (start early — long print); prep copper-shield accent piece. FILM	Printed shell in hand
20	Aug 21 (Fri)		Final assembly: mount board, battery, speaker, OLED, buttons; secure wiring; label donor parts. FILM	Fully assembled physical unit
21	Aug 22 (Sat)		Stress test: battery-life run, playback stability, BLE range, buttons/voice/RGB regression pass.	Punch-list of fixes identified

22	Aug 23 (Sun)	Metrics & Docs	Weigh harvested parts → finalize waste-diverted metrics; shoot hero & before/after photos.	Numbers & photos ready for §8	
23	Aug 24 (Mon)		Write build docs: BOM, wiring diagrams, firmware/app architecture, assembly guide; clean up repo for open-source publish.	Docs package drafted	
24	Aug 25 (Tue)	★ Finish line	Edit & publish YouTube video from B-roll already shot; finalize & export documentation package.	Everything done, 4 days early	
25	Aug 26 (Wed)	Buffer	Fresh-eyes review of the full submission (video, docs, device); fix anything that reads wrong.	Submission package finalized	
26	Aug 27 (Thu)		Rehearse the live pitch/demo (see §8.3); print poster + donor-device cards.	Confident, timed pitch run	
27	Aug 28 (Fri)		Full dry run with charged spares; pack event kit (see §9).	Kit packed, nothing left to chance	
28	Aug 29 (Sat)		Event day	Arrive early, set up, present.	—

7. Documentation & YouTube Video Plan

7.1 Documentation package (compiled Day 23–24)

- ☐ Bill of materials (buy list + harvest list with donor devices)
- ☐ Wiring diagram (the block diagram + pin tables from §3)
- ☐ Firmware architecture & source (commented, open-source repo)
- ☐ Mobile app architecture & source (open-source repo)
- ☐ Photo-illustrated assembly guide
- ☐ Waste-diverted impact sheet (§8.1)
- ☐ Donor-device cards ("this speaker came from a 2014 ...")
- ☐ Open-source license note (MIT for code, CERN-OHL or CC-BY-SA for hardware design)
- ☐ QR code on the case linking to the repo/build guide

7.2 YouTube video — shoot as you build, edit on Day 24

Key move: the days flagged **FILM** in §6 are when you actually capture footage (harvesting, first sound, assembly). Day 24 is *editing*, not filming from scratch — trying to recreate "the moment it first played music" after the fact never looks as good as the real take.

Target length: 2–3 minutes. **Structure:**

1. **Hook (0:00–0:10):** "This speaker was headed for a landfill. Now it plays my music over Bluetooth."
2. **Problem (0:10–0:25):** e-waste stat, why TechMaster's theme matters
3. **Harvest montage (0:25–0:50):** opening the dead laptop/modem/remote, pulling parts (Day 1–2 footage)
4. **Build montage (0:50–1:30):** wiring, first sound, OLED lighting up, 3D print time-lapse (Day 4,9,19,20 footage)
5. **Feature demo (1:30–2:10):** app file transfer, voice control, RGB visualizer
6. **Impact numbers (2:10–2:30):** the waste-diverted metric card, full screen
7. **Close / CTA (2:30–2:45):** "It's open-source — link in the description" + QR code on screen

8. Winning Strategy — Metrics, Display & Judge Pitch

8.1 Waste-diverted metric card (fill in real weighed numbers on Day 22)

Metric	Placeholder — replace with your measured value
E-waste diverted (speaker + battery + buttons + wiring)	___ g
Plastic bottles' worth of filament used in the case	___ bottles (rPETG typically ~10–12g/bottle equivalent)
Lithium cells rescued from landfill	1 × 3.7V cell
Estimated build cost (excluding donor devices)	\$___ (sum of \$2.2)

8.2 Exploded-view display

- Design the enclosure with a clear/translucent panel so judges can see the salvaged speaker, battery, and copper accent without disassembly.
- Set a small card or photo next to the unit for each donor device ("Salvaged from: dead 2014-era laptop / broken modem router / scrapped TV remote").
- Print the QR code (linking to the open-source repo) directly on the case.

8.3 60-second judge pitch (draft — rehearse & personalize on Day 26)

"Every year, tons of laptops, routers, and phones die — not because their parts are broken, but because nobody bothers to save them. This is e-Pod. Its speaker came from a dead laptop. Its battery came from a broken modem. Its buttons came from a scrapped remote control. And its case is 3D-printed from recycled plastic bottles. Inside, it runs on a low-power ESP32-S3, stores music on a salvaged SD reader, and syncs wirelessly over Bluetooth — so you never plug in a cable. It even takes offline voice commands: 'Hey e-Pod, next track.' [demo] And this RGB ring pulses to the beat, powered by a battery that would otherwise be leaking toxic metals in a landfill. In total, e-Pod diverted [X] grams of e-waste and the equivalent of [Y] plastic bottles — and the entire blueprint is open-source, so anyone with a soldering iron and a dead laptop can

build their own. e-Pod doesn't just play music. It proves landfills are full of working parts — we just have to look."

9. Event-Day Checklist

- ☐ e-Pod device, fully charged
- ☐ Backup MicroSD card pre-loaded with a tested playlist
- ☐ Spare charged battery / charging cable + power bank
- ☐ Phone with app installed, BLE tested that morning
- ☐ Backup demo video (in case live BLE demo flakes on venue Wi-Fi/RF noise)
- ☐ Printed poster + donor-device cards
- ☐ QR-code card / sticker linking to the open-source repo
- ☐ Business-card-style handout with the waste-diverted metrics
- ☐ Small toolkit (screwdriver, spare wire, tape) for last-minute fixes

e-Pod — TechMaster "Masters of Recycling" build plan · generated 2026-08-01 · adjust dates/assumptions in the callout box at the top if your real start date or team size changes.